Python Engineering - EX22

WEEK 04 | CLASS 14 - File I/O | Vertical State Persistence

Objetivo: generar una pequena trayectoria con un objeto `VerticalState`, guardarla en un archivo CSV, leerla de nuevo y verificar que los datos recuperados coinciden con los originales.

> Por primera vez nuestros resultados sobreviviran al final del programa: pasaremos de datos que viven en RAM a datos persistentes guardados en disco.

Ruta exacta

```
01_python_bootcamp/
`-- week_04_oop_files_testing/
    `-- class_14_file_io/
        |-- notes/
        `-- exercises/
            `-- ex22_vertical_state_file_io/
                |-- vertical_state.py
                |-- file_io.py
                |-- main.py
                `-- data/
                    |-- vertical_trajectory.csv
                    `-- run_log.txt
```

Arquitectura del ejercicio

```
vertical_state.py
    defines the engineering object
            |
            v
main.py ---- creates/propagates trajectory
            |
            v
file_io.py - writes/reads files
            |
            v
data/vertical_trajectory.csv
```

Valores de referencia

Parametro	Valor
Initial state	h=100.0 m, v=20.0 m/s, t=0.0 s, g=9.81 m/s^2
dt	0.1 s
n_steps	5
Number of saved rows	6 (initial state + 5 propagated states)
Trajectory shape	(6, 3) with columns [h, v, t]
Final Euler state	h=109.019 m, v=15.095 m/s, t=0.5 s approx.
Round-trip check	np.allclose(original, loaded) -> True

BLOCK A - Reutilizar VerticalState

Copia tu clase limpia de EX21 a `vertical_state.py`. No necesitamos los prints de BLOCK dentro de la clase.

```
import numpy as np

class VerticalState:
    def __init__(self, h, v, t=0.0, g=9.81):
        self.h = h
        self.v = v
        self.t = t
        self.g = g

    def state_vector(self):
        return np.array([self.h, self.v, self.t])

    def euler_step(self, dt):
        self.h = self.h + self.v * dt
        self.v = self.v - self.g * dt
        self.t = self.t + dt
```

BLOCK B - Crear una ruta robusta con pathlib

En `main.py` usaremos la carpeta en la que vive el propio script como referencia, no la carpeta desde la que casualmente hayamos abierto la terminal.

```
from pathlib import Path

base_dir = Path(__file__).resolve().parent
data_dir = base_dir / 'data'
data_dir.mkdir(exist_ok=True)

trajectory_file = data_dir / 'vertical_trajectory.csv'
log_file = data_dir / 'run_log.txt'
```

`__file__` representa el archivo Python actual. `.parent` obtiene su carpeta y el operador `/` de Path construye rutas sin concatenar strings manualmente.

BLOCK C - Generar la trayectoria en memoria

```
earth_state = VerticalState(
    h=100.0,
    v=20.0
)

dt = 0.1
n_steps = 5

trajectory_rows = [earth_state.state_vector()]

for _ in range(n_steps):
    earth_state.euler_step(dt)
    trajectory_rows.append(earth_state.state_vector())

trajectory = np.array(trajectory_rows)
```

Resultado esperado:

```
trajectory.shape
# (6, 3)

# approximately:
[[100.0000, 20.0000, 0.0],
 [102.0000, 19.0190, 0.1],
 [103.9019, 18.0380, 0.2],
 [105.7057, 17.0570, 0.3],
 [107.4114, 16.0760, 0.4],
 [109.0190, 15.0950, 0.5]]
```

BLOCK D - file_io.py: guardar CSV

Ahora escribimos una funcion normal de I/O. No es un metodo de VerticalState porque guardar un CSV no es comportamiento fisico del estado; es responsabilidad del modulo de entrada/salida.

```
def save_trajectory_csv(file_path, trajectory):

    with open(file_path, 'w', encoding='utf-8') as file:
        file.write('h_m,v_m_s,t_s\n')

        for row in trajectory:
            h = row[0]
            v = row[1]
            t = row[2]

            file.write(
                f'{h:.6f},{v:.6f},{t:.6f}\n'
            )
```

Modo 'w': abre el archivo para escritura. Si ya existe, su contenido anterior se reemplaza. El bloque `with` se encarga de cerrar correctamente el archivo.

BLOCK E - Comprobar visualmente el archivo

Despues de ejecutar el guardado, abre `data/vertical_trajectory.csv`. Deberia tener una estructura parecida a:

```
h_m,v_m_s,t_s
100.000000,20.000000,0.000000
102.000000,19.019000,0.100000
103.901900,18.038000,0.200000
105.705700,17.057000,0.300000
107.411400,16.076000,0.400000
109.019000,15.095000,0.500000
```

BLOCK F - file_io.py: leer el CSV

```
import numpy as np

def load_trajectory_csv(file_path):

    rows = []

    with open(file_path, 'r', encoding='utf-8') as file:
        header = file.readline()

        for line in file:
            clean_line = line.strip()

            if clean_line == '':
                continue

            values = clean_line.split(',')

            h = float(values[0])
            v = float(values[1])
            t = float(values[2])

            rows.append([h, v, t])

    return np.array(rows)
```

Observa el viaje de los datos:

```
file text
    -> line string
    -> strip()
    -> split(',')
    -> three strings
    -> float(...)
    -> numerical row
    -> np.array(rows)
```

BLOCK G - Validar el round trip

```
save_trajectory_csv(trajectory_file, trajectory)

loaded_trajectory = load_trajectory_csv(trajectory_file)

same_shape = trajectory.shape == loaded_trajectory.shape
same_values = np.allclose(
    trajectory,
    loaded_trajectory
)

print(same_shape)
print(same_values)
```

Esperamos `True` y `True`. A esto lo llamaremos **round-trip validation**: datos -> archivo -> datos reconstruidos.

BLOCK H - Practicar modo append 'a'

Añadiremos un segundo archivo, `run_log.txt`, para guardar un resumen de cada ejecucion sin borrar los anteriores.

```
def append_run_log(file_path, message):

    with open(file_path, 'a', encoding='utf-8') as file:
        file.write(message + '\n')

final_state = trajectory[-1]

message = (
    f'steps={n_steps}, dt={dt}, '
    f'final_h={final_state[0]:.6f}, '
    f'final_v={final_state[1]:.6f}, '
    f'final_t={final_state[2]:.6f}'
)

append_run_log(log_file, message)
```

Cada nueva ejecucion añadira una linea. Esto permite ver claramente la diferencia entre `'w'` y `'a'`.

BLOCK I - Imports y main.py

```
import numpy as np
from pathlib import Path

from vertical_state import VerticalState
from file_io import (
    save_trajectory_csv,
    load_trajectory_csv,
    append_run_log
)

def main():
    # 1. Build paths
    # 2. Create VerticalState object
    # 3. Generate trajectory
    # 4. Save CSV
    # 5. Load CSV
    # 6. Validate round trip
    # 7. Append run log
    # 8. Print compact summary

if __name__ == '__main__':
    main()
```

FINAL VALIDATION

```
Explain in comments:
- What is file I/O?
- What is the difference between RAM data and persistent file data?
- What does open() return?
- What does mode 'w' do if the file already exists?
- What is the difference between 'w', 'r' and 'a'?
- Why do we use 'with open(...) as file:'?
- What does file.write() receive?
- Why do numerical values become text when written to this CSV?
- What does file.readline() do in our loader?
- Why do we call strip() before split(',')?
- What does split(',') return?
- Why do we use float() after split()?
- What does Path(__file__).resolve().parent represent?
- Why is file_io.py separated from vertical_state.py?
- What is a round-trip validation?
- Why is np.allclose() more appropriate than exact float equality here?
```

Criterio de cierre

EX22 estara completado cuando puedas explicar el flujo completo **objeto -> trayectoria en RAM -> texto CSV -> lectura -> numeros -> array NumPy**, y entiendas por que las responsabilidades se reparten entre `vertical_state.py`, `file_io.py` y `main.py`.